

[Function Specification Document] Loan Application & Disbursement

Aug 24, 2026

Target Release	Aug 26, 2026
Document Status	In progress ▾
Document owner	Haikal Raihansyah
UI/UX	-
Front End	-
Back End	-
QA	-

Related Document

Put the related documents, e.g. docs, sheets, papers, miro, figma, etc.

Document	Link	Notes
Test Technical Analyst	☰ Test Integration Anal...	
[FSD] Loan Application	☰ [TED] Loan Applicati...	
Flow Diagram	https://whimsical.com/muhammad-haikal-raihansyah/fintech-lending-app-architecture-flow-KXDSHwwLoiuHWdhY2hTaq5	
Github	https://github.com/haikalraihansyah/integration-analyst-test.git	

1. Document Overview

1.1 Purpose

Dokumen ini menjelaskan kebutuhan fungsional untuk pengembangan fitur **Online Loan Application** pada mobile application PT. XYZ.

Sistem memungkinkan customer untuk melakukan proses pengajuan pinjaman secara digital, mulai dari login, KYC, pengisian data pengajuan, proses verifikasi, credit assessment, persetujuan limit, pemilihan nominal dan tenor, hingga proses pencairan dana.

Sistem juga terintegrasi dengan beberapa external service dan **Core Banking System** melalui **SOA / Integration Layer**.

1.2 Scope

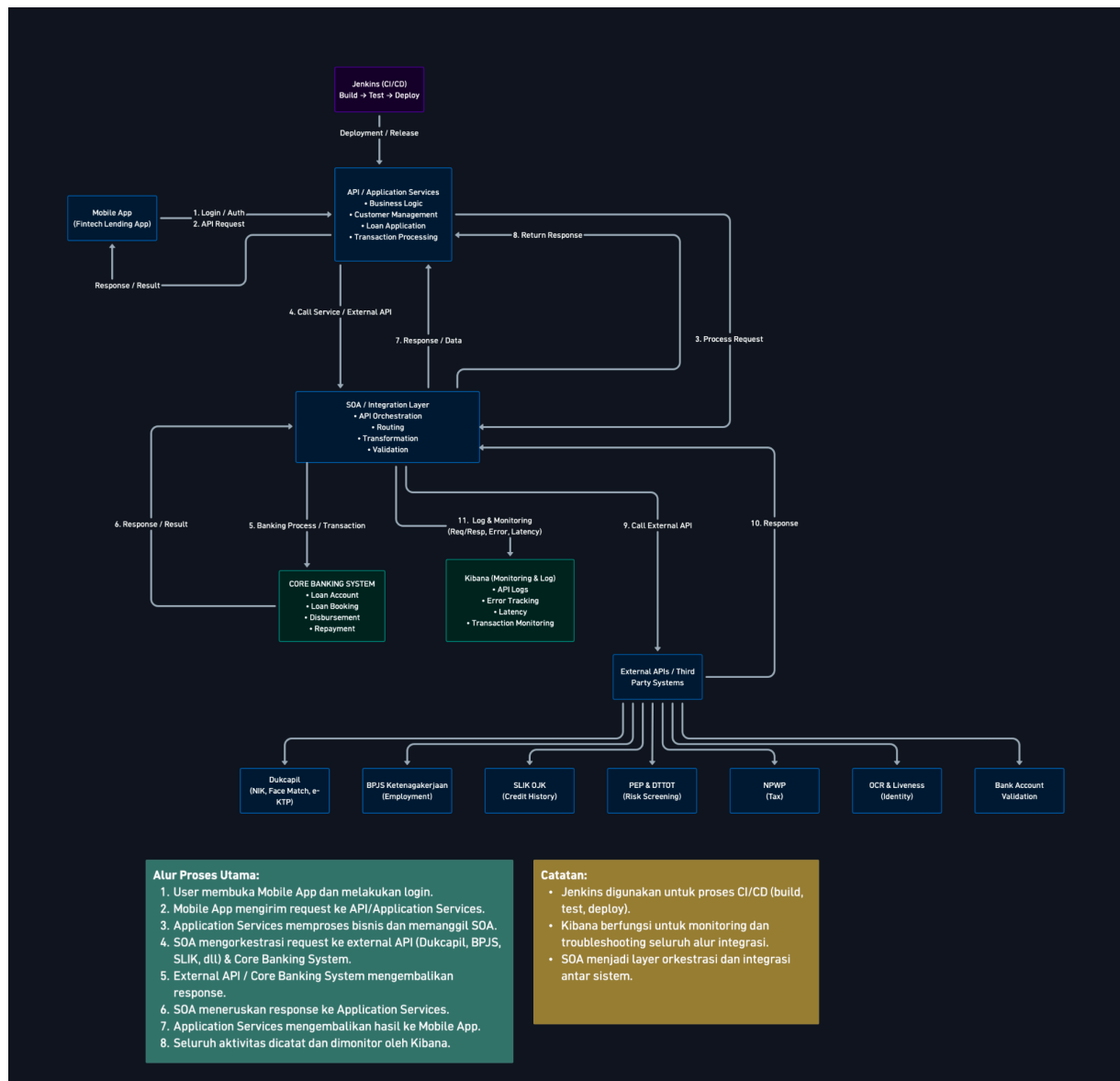
Scope utama sistem meliputi:

1. Customer Authentication
2. KYC Verification
3. Loan Application
4. Customer & Credit Data Verification
5. Manual Credit Review
6. Loan Approval & Limit Assignment
7. Loan Booking
8. Disbursement
9. Loan Activation
10. Logging & Monitoring
11. CI/CD Deployment

2. System Architecture Overview

Arsitektur pada diagram menggunakan pendekatan service-based integration. Mobile application tidak diarahkan untuk mengakses Core Banking System secara langsung. Request dari aplikasi diproses oleh service/backend dan diteruskan melalui integration layer sebelum mencapai sistem core.

2.1 Architecture Diagram



2.2 Architecture Components

Component	Primary Function	Integration Role
Mobile Application	Channel yang digunakan customer untuk mengakses layanan pinjaman.	API consumer; mengirim request dan menerima response.
Service Worker / Backend Service	Menjalankan business logic dan orchestration.	Menerima request dari mobile app dan memanggil service downstream.
SOA / Integration Layer	Menyediakan service dan menjadi abstraction layer antar sistem.	Mengatur komunikasi antara backend dan sistem internal.
Core Banking System	Sistem utama untuk data/account dan proses finansial.	Downstream system yang menyediakan atau memproses data finansial.
OpenAPI 3.1	Mendokumentasikan API contract.	Referensi bersama untuk request, response, endpoint, dan schema.
Jenkins	Automation untuk build, test, dan deployment.	Mendukung delivery service/API secara konsisten.
Kibana	Visualisasi dan analisis log.	Monitoring, investigation, dan troubleshooting.

Catatan: Istilah Service Worker pada diagram diperlakukan sebagai backend/service component.

3. End-to-End Functional Flow

Sebagai contoh functional flow, digunakan skenario customer melakukan pengajuan pinjaman melalui mobile application. Flow ini menunjukkan titik integrasi utama dan bukan merupakan detail proses credit approval karena logic tersebut tidak tersedia pada diagram.

3.1 Loan Application Flow

- Customer mengisi data pengajuan pinjaman pada mobile application.
- Mobile application melakukan validasi dasar terhadap input dan mengirim request ke backend service melalui API.
- Backend menerima request, melakukan authentication/authorization check dan business validation.

- Backend memanggil service pada integration/SOA layer untuk kebutuhan data atau proses yang berada di sistem internal.
- Integration layer meneruskan request ke Core Banking System menggunakan interface yang disediakan.
- Core Banking memproses request dan mengembalikan response.
- Integration layer meneruskan response ke backend dengan format yang sesuai dengan contract API.
- Backend mengembalikan response kepada mobile application.
- Mobile application menampilkan status pengajuan kepada customer.
- Log dari setiap service dicatat untuk kebutuhan monitoring dan troubleshooting.

3.2 Functional Flow Summary

Step	System	Action	Expected Result
1	Mobile App	Submit loan application	Request terbentuk.
2	Backend Service	Validate and process request	Request valid dan siap diteruskan.
3	SOA / API Layer	Route service request	Request diteruskan ke downstream system.
4	Core Banking	Process financial/customer data	Response berhasil atau error dikembalikan.
5	Backend Service	Process downstream response	Response sesuai contract.
6	Mobile App	Display application status	Customer menerima status proses.

4. API Functional Specification

API menjadi contract utama antara mobile application dan backend service. OpenAPI 3.1 dapat digunakan untuk mendokumentasikan contract tersebut sehingga mobile, backend, QA, dan integration team menggunakan referensi yang sama.

4.1 Example Endpoint

Attribute	Specification
Endpoint	POST /loan-applications
Purpose	Membuat pengajuan pinjaman baru.
Consumer	Mobile Application
Provider	Backend / Loan Service
Content-Type	application/json
Authentication	Authenticated API request; mechanism perlu ditentukan pada technical design.
Response	Application ID dan status pengajuan.

4.2 Example Request

POST /loan-applications

Content-Type: application/json

```
{  
  "customerId": "CUST001",  
  "loanAmount": 10000000,  
  "tenor": 12  
}
```

4.3 Example Response

HTTP/1.1 201 Created

```
{  
  "applicationId": "APP001",  
  "status": "SUBMITTED"  
}
```

4.4 Error Response

HTTP Status	Example Condition	Expected Handling
400	Invalid request / missing mandatory field	Return validation message; request tidak diteruskan.
401	Unauthenticated request	Reject request.
403	Insufficient permission	Reject request and log security event.
404	Resource/customer tidak ditemukan	Return functional error.

409	Duplicate submission / conflict	Return existing transaction or conflict response.
500	Internal service error	Log error and return generic error message.
502/503	Downstream service unavailable	Handle as integration failure; retry sesuai policy jika aman.
504	Downstream timeout	Return timeout response and log correlation ID.

5. Data & Integration Requirements

5.1 Data Mapping

Integration layer perlu memiliki mapping yang jelas antara data dari mobile/backend dengan field yang dibutuhkan oleh downstream system. Mapping harus disepakati sebelum development agar perubahan nama field, format, atau tipe data tidak menimbulkan error pada runtime.

Source Field	Target Field	Example	Validation
customerId	customerId / customerNumber	CUST001	Mandatory; format sesuai customer master.
loanAmount	requestedAmount	10000000	Numeric; harus > 0.
tenor	tenorMonths	12	Integer; harus berada dalam range produk.
applicationId	externalReference	APP001	Unique untuk setiap pengajuan.

Assumption: Field mapping di atas merupakan contoh functional mapping dan bukan mapping resmi Core Banking PT. XYZ.

5.2 Data Consistency

- Format tanggal, nominal, currency, dan numeric field harus didefinisikan secara konsisten.
- Field mandatory dan optional harus ditentukan pada API contract.
- Identifier transaksi harus dapat digunakan untuk tracing end-to-end.

- Response dari downstream system perlu ditransformasikan ke format yang dipahami oleh consumer.

6. Security Requirements

Karena sistem digunakan untuk layanan finansial, security menjadi bagian dari integration requirement. Detail mechanism perlu ditentukan oleh security dan architecture team, tetapi minimal requirement berikut perlu dipertimbangkan.

- Semua komunikasi antar service menggunakan encrypted channel seperti HTTPS/TLS.
- API harus memiliki authentication dan authorization sesuai tipe consumer.
- Credential, token, dan sensitive data tidak boleh ditulis secara plain text pada log.
- Akses ke Core Banking dibatasi hanya untuk service yang membutuhkan.
- Request dan transaction identifier harus dapat diaudit tanpa mengekspos data sensitif customer.
- Input dari client harus divalidasi di server dan tidak hanya mengandalkan validasi pada mobile application.

7. Error Handling & Reliability

Integration layer perlu membedakan functional error dengan technical error. Hal ini penting agar customer mendapatkan pesan yang sesuai dan tim operasional dapat mengetahui sumber masalah.

Scenario	Classification	Handling
Invalid customer data	Functional	Return validation/business error.
Insufficient service permission	Security	Reject request and record event.
Core Banking timeout	Technical	Apply timeout policy; retry only when operation is safe.
Connection failure	Technical	Retry based on defined policy, then return controlled error.
Duplicate request	Functional/Technical	Use idempotency/reference ID to avoid duplicate transaction.

Unexpected response schema	Integration	Reject/handle safely and raise alert for investigation.
----------------------------	-------------	---

7.1 Timeout & Retry

Timeout value, retry count, dan retry interval harus ditentukan berdasarkan SLA masing-masing downstream system. Tidak semua transaksi aman untuk di-retry. Untuk transaksi yang dapat menyebabkan duplicate financial transaction, mekanisme idempotency atau unique transaction reference perlu digunakan.

8. Monitoring & Troubleshooting

Kibana pada diagram dapat digunakan untuk melihat dan menganalisis log dari service yang terlibat. Agar troubleshooting end-to-end lebih mudah, setiap request sebaiknya memiliki correlation ID yang diteruskan dari mobile/backend sampai downstream service.

Monitoring Item	Purpose
Correlation ID	Tracing satu request antar service.
Request/Response status	Melihat keberhasilan atau kegagalan transaksi.
Response time	Mengidentifikasi latency atau timeout.
Error code/message	Menentukan sumber kegagalan.
Service availability	Mengetahui service yang down atau tidak responsif.
Transaction volume	Melihat perubahan traffic dan potensi anomaly.

8.1 Example Troubleshooting Flow

- Ambil application ID atau correlation ID dari transaksi yang gagal.
- Cari ID tersebut pada log backend/service.
- Periksa apakah request berhasil diteruskan ke SOA/integration layer.
- Periksa response atau error dari Core Banking.
- Identifikasi apakah masalah termasuk validation, authentication, timeout, connectivity, atau application error.
- Jika diperlukan, koordinasikan issue dengan owner sistem terkait berdasarkan titik kegagalan.

9. CI/CD and Deployment

Jenkins dapat digunakan untuk mengotomatisasi proses build, testing, dan deployment pada service/API. Tujuannya adalah memastikan perubahan yang masuk telah melalui quality gate sebelum digunakan pada environment berikutnya.

Source Code → Jenkins → Build → Automated Test → Deploy → Monitoring

Stage	Expected Activity
Build	Compile/package application atau service.
Unit Test	Menjalankan test untuk business logic.
Integration Test	Memastikan service dapat berkomunikasi dengan dependency.
Quality/Security Check	Menjalankan quality atau security validation sesuai pipeline.
Deploy	Deploy ke environment yang ditentukan.
Post-Deployment	Monitoring log, health, dan error rate.

10. Functional Requirements

ID	Requirement	Priority
FR-001	Mobile application harus dapat mengirim pengajuan pinjaman melalui API.	High
FR-002	Backend harus memvalidasi mandatory field sebelum request diteruskan.	High
FR-003	Backend/integration layer harus dapat meneruskan request ke downstream service.	High
FR-004	Response dari downstream harus diproses dan dikembalikan dalam API contract yang disepakati.	High
FR-005	API harus memiliki documented request, response, dan error schema.	High

FR-006	Setiap transaksi harus dapat ditelusuri menggunakan unique reference/correlation ID.	High
FR-007	Technical error harus menghasilkan controlled response dan tercatat pada logging system.	High
FR-008	Service harus dapat dimonitor melalui log/monitoring platform.	Medium
FR-009	Deployment service harus dapat dijalankan melalui CI/CD pipeline.	Medium

11. Non-Functional Requirements

Category	Requirement
Security	Communication, authentication, authorization, dan protection terhadap sensitive data harus diterapkan.
Performance	Response time harus memenuhi SLA yang disepakati dengan system owner.
Availability	Service kritikal harus memiliki availability target sesuai business requirement.
Scalability	Backend/API harus dapat menangani peningkatan jumlah request.
Traceability	Transaction harus dapat ditelusuri end-to-end menggunakan correlation/reference ID.
Maintainability	API contract dan integration documentation harus selalu diperbarui.
Observability	Log, error, latency, dan service status harus dapat dimonitor.

12. Integration Testing Considerations

Integration testing dilakukan untuk memastikan komunikasi antar komponen berjalan sesuai contract. Testing tidak hanya mencakup positive scenario, tetapi juga failure scenario.

Test Scenario	Expected Result
Valid loan application	Request diterima dan application ID dikembalikan.
Missing mandatory field	API mengembalikan 400 dengan validation error.
Invalid authentication	API mengembalikan 401.
Customer not found	Functional error dikembalikan sesuai contract.
Core Banking timeout	Request menghasilkan controlled timeout/error response dan tercatat di log.
Duplicate submission	System tidak membuat transaksi duplicate.
Unexpected downstream error	Error ditangani dan tidak mengekspos detail internal kepada customer.
High response latency	Latency dapat ditemukan melalui monitoring/log.

13. Conclusion

Arsitektur pada diagram menunjukkan pendekatan integrasi berbasis service yang memisahkan mobile application dari Core Banking System. Pemisahan tersebut memungkinkan business logic dan akses ke sistem core dikontrol melalui backend serta integration layer.

Untuk implementasi layanan pinjaman online, keberhasilan integrasi tidak hanya ditentukan oleh koneksi antar sistem, tetapi juga oleh kejelasan API contract, data mapping, security, error handling, monitoring, dan traceability. OpenAPI 3.1 dapat menjadi acuan contract API, Jenkins mendukung proses delivery, dan Kibana mendukung monitoring serta troubleshooting. Detail yang belum tersedia pada diagram perlu dikonfirmasi pada technical design sebelum development dimulai.